

# Jaroché — VPS Deployment Guide

---

Step-by-step guide to deploy this project on a Linux VPS (Ubuntu / Debian). Designed to coexist with other apps on the same server — uses ports `3001` and `4001` and creates its own database `jaroche_prod`.

---

## What you'll have at the end

---

- Storefront running at `http://<vps-ip>:3001` (or `https://your-domain` once nginx + SSL are set up)
  - Express API at `http://<vps-ip>:4001/api/*` (proxied internally — you don't usually hit this directly)
  - Postgres database `jaroche_prod` with seeded admin
  - Both services managed by **pm2**, auto-restart on crash, auto-resurrect on reboot
  - Admin login: `admin@jaroche.dev` / `Password123!`
- 

## Prerequisites

---

The VPS must have:

| Software       | Why                 | Install on Ubuntu/Debian                                         |                                                                            |
|----------------|---------------------|------------------------------------------------------------------|----------------------------------------------------------------------------|
| Node 20+       | runs the app        | <code>`curl -fsSL https://deb.nodesource.com/setup_20.x \</code> | <code>sudo -E bash - &amp;&amp;<br/>sudo apt install -y<br/>nodejs`</code> |
| PostgreSQL 14+ | database            | <code>sudo apt install -y postgresql</code>                      |                                                                            |
| nginx          | reverse proxy + SSL | <code>sudo apt install -y nginx</code>                           |                                                                            |
| certbot        | free SSL certs      | <code>sudo apt install -y certbot python3-certbot-nginx</code>   |                                                                            |
| git            | clone the repo      | <code>sudo apt install -y git</code>                             |                                                                            |
| openssl        | generates secrets   | usually preinstalled                                             |                                                                            |

`pm2` is installed automatically by the deploy script if it's missing.

Verify each is available:

```
node -v          # v20.x or higher
npm -v
psql --version   # 14+
nginx -v
git --version
```

## Step 1 — Clone the repository

Pick a directory and clone into it. Example for `/var/www/carl/jaroche` :

```
sudo mkdir -p /var/www/carl
cd /var/www/carl
sudo git clone https://github.com/Toniownn/jaroche.git
cd jaroche
```

If you already cloned and want the latest deploy script:

```
cd /var/www/carl/jaroche
git fetch origin
git checkout add-deploy-script      # or: git pull origin main once merged
ls -l deploy.sh
```

---

## Step 2 — Run the deploy script

---

This is the only command you really need. It runs end-to-end and is safe to re-run (idempotent):

```
cd /var/www/carl/jaroche
bash deploy.sh
```

### What it does, in order

1. **Preflight** — checks `node`, `npm`, `psql`, `openssl`; installs `pm2` if missing 2. **Postgres** — creates role `jaroche` and database `jaroche_prod` if absent; generates a random password the first time 3. **.env** — writes a `.env` file at the repo root with: - `DATABASE_URL` (pointing at the new DB) - random `JWT_ACCESS_SECRET` and `JWT_REFRESH_SECRET` - placeholder Stripe keys (replace later when you have them) - `PORT=4001` 4. **Next.js config patch** — updates `client/next.config.ts` so the storefront proxies `/api/*` to the server on port 4001 (no collisions with other apps) 5. **npm install** in all three workspaces 6. **Build** — `shared` → `server` → `client` 7. **Prisma** — `migrate deploy + db seed` 8. **pm2** — starts two named processes: - `jaroche-server` on `127.0.0.1:4001` - `jaroche-client` on `127.0.0.1:3001` Then saves the process list and enables `resurrect-on-reboot`. 9. **Nginx sample** — writes `nginx-jaroche.conf` to the repo root, ready to copy into `/etc/nginx/sites-available/`.

The script prints a summary at the end with follow-up commands.

### Optional environment overrides

If you want different ports or names, set them before running:

```
DOMAIN=jaroche.example.com \
CLIENT_PORT=3001 \
SERVER_PORT=4001 \
DB_NAME=jaroche_prod \
DB_USER=jaroche \
bash deploy.sh
```

---

## Step 3 — Verify it's running

---

```
# pm2 status – both should be "online"
pm2 ls

# storefront responds with HTML (HTTP/1.1 200)
curl -sI http://127.0.0.1:3001 | head -3

# API responds with JSON (60 products)
curl -s http://127.0.0.1:4001/api/products | head -c 200
```

Expected: `pm2 ls` shows `jaroche-client` and `jaroche-server` both **online**, and the API call returns a JSON array starting with `[{"id":"...", "name":"...", .`

---

## Step 4 — Point a domain at it (optional but recommended)

---

### 4a. Update your DNS

In your domain registrar, create an **A record** pointing `jaroche.your-domain.com` to the VPS IP.

Wait for DNS to propagate:

```
dig +short jaroche.your-domain.com    # should return your VPS IP
```

### 4b. Install the nginx vhost

```
# edit the sample to use your real domain
sudo sed -i 's/server_name _;/server_name jaroche.your-domain.com;/' \
/var/www/carl/jaroche/nginx-jaroche.conf

# copy into nginx config and enable
sudo cp /var/www/carl/jaroche/nginx-jaroche.conf /etc/nginx/sites-available/jaroche.conf
sudo ln -sf /etc/nginx/sites-available/jaroche.conf /etc/nginx/sites-enabled/jaroche.conf

# test syntax, then reload
sudo nginx -t
sudo systemctl reload nginx
```

You should now be able to hit `http://jaroche.your-domain.com` and see the storefront. (HTTP only at this point — next step adds HTTPS.)

#### 4c. Add HTTPS with certbot (one command)

```
sudo certbot --nginx -d jaroche.your-domain.com
```

Certbot will modify the nginx config in place to redirect HTTP → HTTPS and install a Let's Encrypt cert that auto-renews. Done.

---

## Step 5 — Wire up Stripe (later, when you have keys)

---

The deploy script writes Stripe placeholders. The site works without them — only the checkout button will fail. When you're ready:

```
cd /var/www/carl/jaroche
nano .env
# replace these two lines with your real keys:
# STRIPE_SECRET_KEY=sk_live...
# STRIPE_WEBHOOK_SECRET=whsec...

# restart the server so it picks up the new keys
pm2 restart jaroche-server
```

Then in the Stripe dashboard, set the webhook endpoint to:

```
https://jaroche.your-domain.com/api/stripe/webhook
```

---

## Useful pm2 commands

---

```
pm2 ls                # list services + status
pm2 logs jaroche-server # tail server logs (Ctrl-C to exit)
pm2 logs jaroche-client # tail client logs
pm2 logs jaroche-server --lines 200 # last 200 lines
pm2 restart jaroche-server # restart after .env change
pm2 restart all          # restart everything
pm2 monit                # live dashboard
pm2 save                 # save current process list (after changes)
```

## Updating to a new version

---

Once you make changes and push to `main` :

```
cd /var/www/carl/jaroche
git checkout main
git pull origin main
bash deploy.sh          # safe to re-run – won't regenerate .env or DB password
```

`deploy.sh` re-runs migrations, rebuilds, and restarts pm2. Existing data and secrets are preserved.

---

## Troubleshooting

---

### pm2 services keep restarting

Tail the logs to find the error:

```
pm2 logs jaroche-server --lines 100
pm2 logs jaroche-client --lines 100
```

Most common causes: - **Missing env var** — check `.env` has all required keys (run `cat .env`) - **DB unreachable** — verify with `psql "$(grep DATABASE_URL .env | cut -d= -f2- | tr -d \")" -c 'SELECT 1;'` - **Build is stale** — re-run `bash deploy.sh`

## "Port already in use"

Another app is on 3001 or 4001. Override the ports:

```
CLIENT_PORT=3002 SERVER_PORT=4002 bash deploy.sh
```

Remember to update `nginx-jaroche.conf` to match.

## Database password got lost

If you ever lose track of the password but `.env` still has the right `DATABASE_URL`, no action needed — the deploy script reuses the one in `.env`. If `.env` was deleted, regenerate by deleting `.env` and re-running `bash deploy.sh` (it'll generate fresh secrets and reset the DB user password to match).

## Storefront shows blank/loading forever

Most often the API is unreachable. Check:

```
curl -v http://127.0.0.1:4001/api/products  
pm2 logs jaroche-server --lines 50
```

If the server crashed on boot, the migrations might not have applied. Run:

```
cd /var/www/carl/jaroche/server  
npx prisma migrate deploy
```

---

## Uninstall / start over

---

```
# stop services
pm2 delete jaroche-client jaroche-server
pm2 save

# drop database
sudo -u postgres psql -c "DROP DATABASE IF EXISTS jaroche_prod;"
sudo -u postgres psql -c "DROP ROLE IF EXISTS jaroche;"

# remove files
sudo rm -rf /var/www/carl/jaroche
sudo rm -f /etc/nginx/sites-enabled/jaroche.conf /etc/nginx/sites-available/jaroche.conf
sudo systemctl reload nginx
```

## File reference

---

After deployment, these are the files that matter on the VPS:

|                        |                                                     |
|------------------------|-----------------------------------------------------|
| /var/www/carl/jaroche/ |                                                     |
| ├─ deploy.sh           | Deployment script (this guide is built around it)   |
| ├─ DEPLOY.md           | This file                                           |
| ├─ .env                | Secrets – chmod 600, don't commit, don't share      |
| ├─ nginx-jaroche.conf  | Sample nginx vhost – copy into sites-available      |
| ├─ client/             | Next.js storefront – runs via pm2 as jaroche-client |
| ├─ server/             | Express API – runs via pm2 as jaroche-server        |
| └─ shared/             | Shared Zod schemas + types                          |

PM2 state lives at `/root/.pm2/`. Don't delete it — that's what makes the services come back after reboot.